



Python Programming Fundamentals

Scope and Pure Functions

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure Functions
5. Constants and Module-Level Names
6. Scope in Practice



Outline

- 1. Local vs Global Scope**
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure Functions
5. Constants and Module-Level Names
6. Scope in Practice



1. Local vs Global Scope

Variables live in a **scope** — the region of code where they are accessible.

```
message = "I am global"    # global scope
```

```
def show_message():  
    local_msg = "I am local"    # local scope  
    print(message)              # can read global  
    print(local_msg)            # can read local
```

```
show_message()  
print(message)    # OK  
# print(local_msg) # NameError – doesn't exist here
```



1. Local vs Global Scope

Variables live in a **scope** — the region of code where they are accessible.

```
message = "I am global"      # global scope

def show_message():
    local_msg = "I am local"  # local scope
    print(message)           # can read global
    print(local_msg)         # can read local

show_message()
print(message)               # OK
# print(local_msg)          # NameError – doesn't exist here
```

Each function call creates its own **local namespace**. When the function returns, its locals are discarded.



Outline

1. Local vs Global Scope
- 2. The LEGB Rule**
3. Avoiding Global Variables
4. Pure Functions
5. Constants and Module-Level Names
6. Scope in Practice



2. The LEGB Rule

Python searches for names in this order:

Letter	Scope	Example
L	Local — inside current function	x defined in my_func
E	Enclosing — outer function (closures)	Nested functions
G	Global — module level	PI = 3.14159
B	Built-in — Python's built-ins	len, print, range



2. The LEGB Rule



2. The LEGB Rule

Python searches for names in this order:

Letter	Scope	Example
L	Local — inside current function	x defined in my_func
E	Enclosing — outer function (closures)	Nested functions
G	Global — module level	PI = 3.14159
B	Built-in — Python's built-ins	len, print, range

```
x = "global"
```

```
def outer():  
    x = "enclosing"  
    def inner():  
        # x = "local"      # uncomment to use local
```



2. The LEGB Rule

```
    print(x)          # finds "enclosing" (E before G)
inner()

outer()   # enclosing
```



Outline

1. Local vs Global Scope
2. The LEGB Rule
- 3. Avoiding Global Variables**
4. Pure Functions
5. Constants and Module-Level Names
6. Scope in Practice



3. Avoiding Global Variables

Reading a global is fine. **Modifying** a global from inside a function is a code smell.

```
# Bad pattern – hidden global state
```

```
total = 0
```

```
def add_to_total(value):
```

```
    global total
```

```
    total += value          # modifies global – hard to test
```

```
add_to_total(10)
```

```
add_to_total(25)
```

```
print(total)      # 35
```



3. Avoiding Global Variables



3. Avoiding Global Variables

Reading a global is fine. **Modifying** a global from inside a function is a code smell.

```
# Bad pattern – hidden global state
```

```
total = 0
```

```
def add_to_total(value):
```

```
    global total
```

```
    total += value          # modifies global – hard to test
```

```
add_to_total(10)
```

```
add_to_total(25)
```

```
print(total)    # 35
```

Better pattern – pass data in, return data out:

```
def add_values(current_total, value):
```

```
    return current_total + value
```



3. Avoiding Global Variables

```
total = 0
total = add_values(total, 10)
total = add_values(total, 25)
print(total)    # 35
```

Now the function is **testable** and has **no side effects**.



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
- 4. Pure Functions**
5. Constants and Module-Level Names
6. Scope in Practice



4. Pure Functions

A **pure function**:

1. Always produces the same output for the same input
2. Has **no side effects** (doesn't modify external state, no I/O)

Pure – same input always gives same output, no side effects

```
def multiply(a, b):  
    return a * b
```

```
def clamp(value, minimum, maximum):  
    """Constrain value to [minimum, maximum] range."""  
    return max(minimum, min(maximum, value))
```

```
print(clamp(150, 0, 100))    # 100  
print(clamp(-5, 0, 100))    # 0  
print(clamp(50, 0, 100))    # 50
```



4. Pure Functions



4. Pure Functions

A **pure function**:

1. Always produces the same output for the same input
2. Has **no side effects** (doesn't modify external state, no I/O)

Pure – same input always gives same output, no side effects

```
def multiply(a, b):  
    return a * b
```

```
def clamp(value, minimum, maximum):  
    """Constrain value to [minimum, maximum] range."""  
    return max(minimum, min(maximum, value))
```

```
print(clamp(150, 0, 100))    # 100  
print(clamp(-5, 0, 100))    # 0  
print(clamp(50, 0, 100))    # 50
```



4. Pure Functions

Impure — has a side effect (printing, reading input, modifying external state):

```
def get_score():          # impure – depends on user input
    return float(input("Score: "))
```

```
def log_result(msg):      # impure – side effect (printing)
    print(msg)
```



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure Functions
- 5. Constants and Module-Level Names**
6. Scope in Practice



5. Constants and Module-Level Names

Constants are module-level names that never change. By convention, use ALL_CAPS.

```
PI = 3.14159265
```

```
GRAVITY = 9.81          # m/s2
```

```
TAX_RATE = 0.23
```

```
MAX_RETRIES = 3
```

```
def circle_area(radius):  
    return PI * radius ** 2
```

```
def circle_circumference(radius):  
    return 2 * PI * radius
```

```
print(f"Area of r=5 circle: {circle_area(5):.2f}")  
print(f"Circumference: {circle_circumference(5):.2f}")
```



5. Constants and Module-Level Names

Constants are fine at module level — they are **not expected to change** and are clearly marked.



Outline

1. Local vs Global Scope
2. The LEGB Rule
3. Avoiding Global Variables
4. Pure Functions
5. Constants and Module-Level Names
- 6. Scope in Practice**



6. Scope in Practice

```
def calculate_statistics(numbers):  
    """Return mean, min, max for a list of numbers."""  
    if not numbers:  
        return None, None, None  
  
    count = len(numbers)          # local  
    total = sum(numbers)          # local  
    mean = total / count          # local  
    minimum = min(numbers)        # local  
    maximum = max(numbers)        # local  
  
    return mean, minimum, maximum  
  
data = [23, 45, 12, 67, 34, 89, 56]  
avg, lo, hi = calculate_statistics(data)  
print(f"Mean: {avg:.1f}")
```



6. Scope in Practice

```
print(f"Range: {lo} to {hi}")
```

```
# count, total, mean, minimum, maximum don't exist here
```

All intermediate variables are local to the function — clean and encapsulated.



Thanks for Watching - Any questions?

